

## Lab 13 – Implementation of DAG

### Aim

To implement a Directed Acyclic Graph (DAG) for a basic block and identify common subexpressions.

### Algorithm

1. Start the program.
2. Read the number of three-address statements.
3. For each statement  $x = y \text{ op } z$ :
  1. Check whether a node with the same operator and operands already exists.
  2. If it exists, reuse that node (common subexpression).
  3. Otherwise create a new node.
4. Store the result variable with that node.
5. After processing all statements, display the DAG nodes.
6. Stop.

### C Program

```
#include <stdio.h>
#include <string.h>

#define MAX 20

struct Node {
    char op;
    char left;
    char right;
    char result;
};

struct Node dag[MAX];
int count = 0;

int findNode(char op, char left, char right) {
    for (int i = 0; i < count; i++) {
        if (dag[i].op == op &&
            dag[i].left == left &&
            dag[i].right == right)
            return i;
    }
    return -1;
}

int main() {
    int n;
```

```

char stmt[20];

printf("Enter number of statements: ");
scanf("%d", &n);

for (int i = 0; i < n; i++) {
    char res, arg1, arg2, op;

    printf("Enter statement %d (Example: a=b+c): ", i + 1);
    scanf("%s", stmt);

    res = stmt[0];
    arg1 = stmt[2];
    op = stmt[3];
    arg2 = stmt[4];

    int pos = findNode(op, arg1, arg2);

    if (pos == -1) {
        dag[count].op = op;
        dag[count].left = arg1;
        dag[count].right = arg2;
        dag[count].result = res;
        count++;
    } else {
        printf("Common subexpression found for %c=%c%c%c\n",
            res, arg1, op, arg2);
    }
}

printf("\nDAG Representation\n");
printf("Node\tOp\tLeft\tRight\tResult\n");

for (int i = 0; i < count; i++) {
    printf("%d\t%c\t%c\t%c\t%c\n",
        i,
        dag[i].op,
        dag[i].left,
        dag[i].right,
        dag[i].result);
}

return 0;
}

```

## Sample Input

Enter number of statements: 3  
a=b+c

$d=b+c$   
 $e=a+d$

## Sample Output

Common subexpression found for  $d=b+c$

DAG Representation

| Node | Op | Left | Right | Result |
|------|----|------|-------|--------|
| 0    | +  | b    | c     | a      |
| 1    | +  | a    | d     | e      |

## Result

Thus the DAG was implemented successfully and common subexpressions were identified.